

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO THỰC TẬP AI
BÀI 8: LARGE LANGUAGE MODELS FOR
NATURAL LANGUAGE PROCESSING

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN

SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Tổng quan nội dung bài thực hành	1
1.2 Mục tiêu bài thực hành	1
1.3 Phạm vi và dữ liệu sử dụng	1
2. CƠ SỞ LÝ THUYẾT	3
2.1 Large Language Models và xử lý ngôn ngữ tự nhiên.....	3
2.2 Kiến trúc Transformer	4
2.3 Tokenization và biểu diễn văn bản	5
2.4 Mô hình BERT và Fine-tuning	6
2.5 Sentiment Analysis và chỉ số đánh giá	7
2.6 GPT-2 và Text Generation.....	8
2.7 Prompt Engineering và Chatbot.....	10
2.8 Sentence Transformer, FAISS và RAG.....	11
3. QUY TRÌNH THỰC HIỆN.....	13
3.1 Môi trường và thư viện	13
3.2 Chuẩn bị và kiểm tra dữ liệu.....	13
3.3 Tạo Dataset và tokenization.....	14
3.4 Huấn luyện và đánh giá BERT	14
3.5 Thí nghiệm learning rate.....	15
3.6 Sinh văn bản và xây dựng chatbot.....	15
3.7 Xây dựng RAG	15
4. KẾT QUẢ THỰC NGHIỆM.....	16
4.1 Bài 1: Tải dataset IMDB	16
4.2 Bài 2: Hiển thị một số câu trong dataset.....	17
4.3 Bài 3: Tokenize dữ liệu bằng BERT tokenizer.....	18

4.4 Bài 4: Fine-tune mô hình BERT	18
4.5 Bài 5: Tính accuracy trên tập test	20
4.6 Bài 6: Thử thay đổi learning rate	21
4.7 Bài 7: Text generation với GPT-2	23
4.8 Bài 8: Thử các prompt và tham số sinh	24
4.9 Bài 9: Xây dựng chatbot đơn giản	26
4.10 Bài 10: Thử nghiệm RAG với ít nhất 5 documents	28
4.11 Bài 11: Phân tích ưu điểm của RAG so với LLM thuần	30
5. PHÂN TÍCH VÀ THẢO LUẬN	32
5.1 Đánh giá mô hình BERT	32
5.2 Ảnh hưởng của learning rate	32
5.3 Prompt engineering và chất lượng GPT-2	32
5.4 Hạn chế của chatbot.....	33
5.5 Phân tích RAG so với LLM thuần.....	33
5.6 Các lỗi gặp phải và cách khắc phục.....	34
5.7 Đề xuất cải thiện thực nghiệm	34
6. KẾT LUẬN.....	36
TÀI LIỆU THAM KHẢO	37
PHỤ LỤC: MÃ NGUỒN TRỌNG TÂM.....	38
A. Hàm tính chỉ số đánh giá	38
B. Hàm truy xuất tài liệu.....	38
C. Hàm tạo câu trả lời RAG	38

DANH MỤC HÌNH ẢNH

Hình 1: Tổng quan ứng dụng của Large Language Models trong NLP.	3
Hình 2: Kiến trúc Transformer và cơ chế Self-Attention.	5
Hình 3: Quy trình fine-tuning BERT cho bài toán Sentiment Analysis.	7
Hình 4: Quy trình phân loại cảm xúc trên tập dữ liệu IMDB.	8
Hình 5: Cơ chế sinh văn bản tự hồi quy của GPT-2.	9
Hình 6: Sơ đồ hoạt động của chatbot sử dụng GPT-2.	10
Hình 7: Kiến trúc hệ thống Retrieval-Augmented Generation.	12
Hình 8: Kết quả kiểm tra dữ liệu IMDB sau khi tải vào chương trình.	14
Hình 9: Thông tin tập dữ liệu IMDB sau khi tải và các mẫu dữ liệu đầu tiên	16
Hình 10: Kết quả hiển thị một số câu hỏi trong dataset	17
Hình 11: Hiển thị số mẫu train và test	18
Hình 12: Kết quả Fine-tuning mô hình BERT.	19
Hình 13: Kết quả accuracy trên tập test.	20
Hình 14: So sánh Accuracy theo Learning Rate từ notebook	22
Hình 15: So sánh F1-score theo Learning Rate từ notebook.	23
Hình 16: Kết quả sinh văn bản bằng mô hình GPT-2	24
Hình 17: Kết quả thử các prompt và tham số sinh	25
Hình 18: Kết quả xây dựng chatbot đơn giản	27
Hình 19: Kết quả tạo với 7 documents và embedding dimension 384	29
Hình 20: Kết quả truy xuất tài liệu liên quan bằng FAISS trong hệ thống RAG.	29
Hình 21: Kết quả sinh câu trả lời của hệ thống RAG dựa trên tài liệu truy xuất.	30

DANH MỤC BẢNG

Bảng 1: Thống kê dữ liệu IMDB sử dụng trong notebook.....	2
Bảng 2: Môi trường thực nghiệm	13
Bảng 3: Các siêu tham số của mô hình BERT chính.....	14
Bảng 4: Kết quả tải dataset IMDB từ Kaggle Input	16
Bảng 5: Ba mẫu review được hiển thị từ tập train.....	17
Bảng 6: Kết quả quá trình fine-tune BERT	19
Bảng 7: Kết quả đánh giá BERT trên tập test.....	20
Bảng 8: Kết quả dự đoán sentiment cho ba câu kiểm tra	21
Bảng 9: Kết quả thí nghiệm learning rate trên tập dữ liệu rút gọn.....	21
Bảng 10: Kết quả sinh văn bản với năm prompt khác nhau	25
Bảng 11: Quan sát khi thay đổi max_length và temperature.....	26
Bảng 12: Kết quả thử nghiệm mini chatbot.....	27
Bảng 13: Kết quả truy xuất FAISS cho câu hỏi về BERT	29
Bảng 14: So sánh RAG và LLM thuần từ kết quả thực nghiệm.....	31
Bảng 15: Tổng hợp lỗi/cảnh báo trong notebook và hướng khắc phục.....	34

1. GIỚI THIỆU

1.1 Tổng quan nội dung bài thực hành

Large Language Models (LLMs) là các mô hình học sâu được huấn luyện trên lượng lớn dữ liệu văn bản để học biểu diễn ngôn ngữ và thực hiện nhiều nhiệm vụ xử lý ngôn ngữ tự nhiên. Trong bài thực hành số 8, các mô hình tiền huấn luyện được khai thác theo hai hướng: fine-tune BERT cho phân loại cảm xúc và sử dụng GPT-2 cho sinh văn bản. Bài thực hành đồng thời khảo sát prompt engineering, chatbot đơn giản và cơ chế Retrieval-Augmented Generation (RAG).

Notebook được triển khai trên Kaggle với GPU Tesla T4. Bộ dữ liệu IMDB được đọc từ dữ liệu cục bộ, sau đó rút gọn còn 3.000 mẫu huấn luyện và 1.000 mẫu kiểm tra để phù hợp thời gian thực hành. Kết quả, mã nguồn và các biểu đồ trong báo cáo được lấy trực tiếp từ file b-i-8.ipynb.

1.2 Mục tiêu bài thực hành

Hiểu vai trò của mô hình ngôn ngữ tiền huấn luyện trong các nhiệm vụ NLP.

- Xây dựng pipeline dữ liệu IMDB, tokenization bằng bert-base-uncased và fine-tune mô hình BERT cho sentiment analysis.
- Đánh giá mô hình bằng accuracy, precision, recall và F1-score.
- Khảo sát ảnh hưởng của learning rate trên cùng cấu hình huấn luyện rút gọn.
- Thực hiện sinh văn bản bằng GPT-2 và quan sát ảnh hưởng của prompt, max_length và temperature.
- Xây dựng chatbot đơn giản và hệ RAG với SentenceTransformer, FAISS và GPT-2.
- Phân tích ưu điểm, hạn chế, lỗi thực nghiệm và đề xuất hướng cải thiện.

1.3 Phạm vi và dữ liệu sử dụng

Bộ dữ liệu IMDB gồm các bài đánh giá phim bằng tiếng Anh, được gán nhãn Negative (0) hoặc Positive (1). Notebook chọn tối đa 1.500 mẫu cho mỗi lớp trong tập train và 500 mẫu cho mỗi lớp trong tập test, sau đó trộn dữ liệu với random seed 42. Cách chọn này tạo ra hai tập cân bằng, thuận lợi cho việc diễn giải các chỉ số accuracy và F1-score.

Bảng 1: Thống kê dữ liệu IMDB sử dụng trong notebook

Tập dữ liệu	Negative	Positive	Tổng số mẫu	Mục đích
Train	1.500	1.500	3.000	Fine-tune BERT
Test	500	500	1.000	Đánh giá mô hình
Train rút gọn	xấp xỉ cân bằng	xấp xỉ cân bằng	1.000	So sánh learning rate
Test rút gọn	xấp xỉ cân bằng	xấp xỉ cân bằng	300	So sánh learning rate

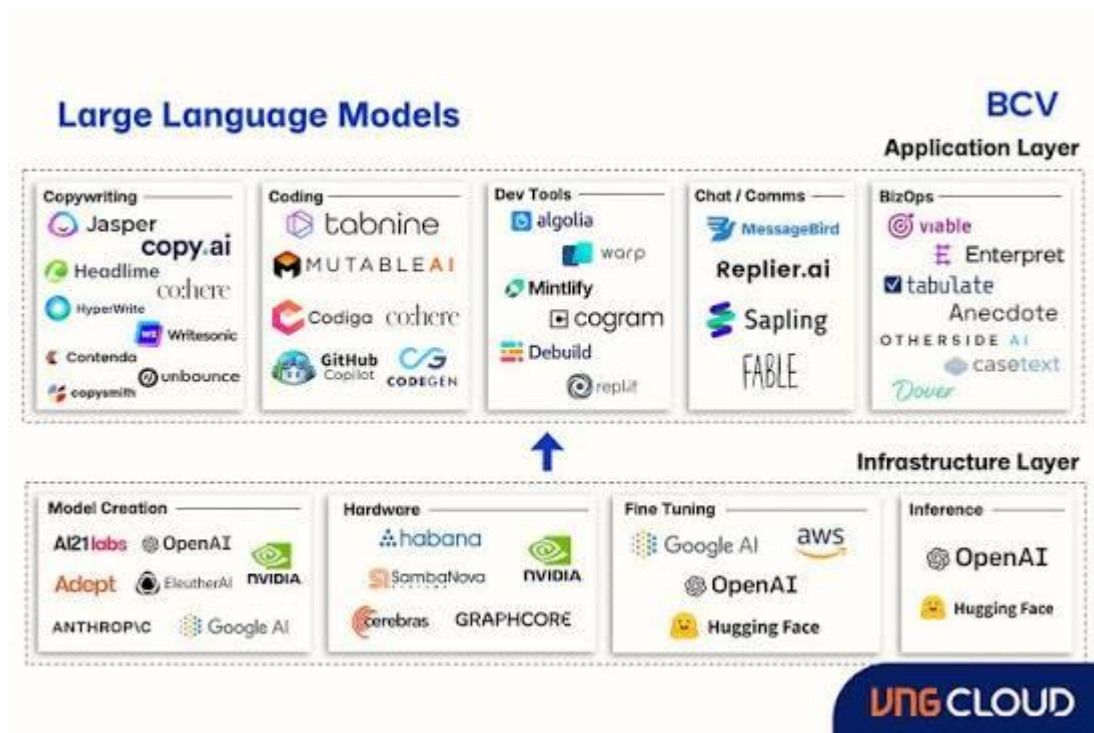
2. CƠ SỞ LÝ THUYẾT

2.1 Large Language Models và xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên, hay Natural Language Processing, là lĩnh vực của trí tuệ nhân tạo giúp máy tính có khả năng hiểu, phân tích và sinh ngôn ngữ của con người. Dữ liệu đầu vào trong NLP thường là văn bản như câu hỏi, bình luận, đánh giá phim, đoạn hội thoại hoặc tài liệu.

Large Language Models, viết tắt là LLMs, là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản rất lớn. Nhờ đó, mô hình có thể học được cấu trúc ngữ pháp, ngữ nghĩa và mối quan hệ giữa các từ trong câu. LLMs được ứng dụng trong nhiều bài toán như phân loại văn bản, phân tích cảm xúc, trả lời câu hỏi, dịch máy, sinh văn bản và xây dựng chatbot.

Trong bài thực hành này, các mô hình và kỹ thuật chính được sử dụng gồm BERT cho bài toán phân tích cảm xúc, GPT-2 cho bài toán sinh văn bản, Prompt Engineering để điều khiển đầu ra và Retrieval-Augmented Generation để kết hợp truy xuất tài liệu với mô hình ngôn ngữ.



Hình 1: Tổng quan ứng dụng của Large Language Models trong NLP.

2.2 Kiến trúc Transformer

Transformer là kiến trúc nền tảng của nhiều mô hình ngôn ngữ lớn hiện nay như BERT, GPT, T5 và LLaMA. Khác với RNN hoặc LSTM xử lý dữ liệu theo thứ tự tuần tự, Transformer sử dụng cơ chế Attention để mô hình có thể xem xét toàn bộ câu cùng lúc. Điều này giúp mô hình học được mối quan hệ xa giữa các từ và tăng tốc quá trình huấn luyện.

Kiến trúc Transformer gồm hai phần chính là Encoder và Decoder. Encoder có nhiệm vụ mã hóa thông tin đầu vào, thường dùng trong các mô hình hiểu văn bản như BERT. Decoder có nhiệm vụ sinh chuỗi đầu ra, thường dùng trong các mô hình sinh văn bản như GPT.

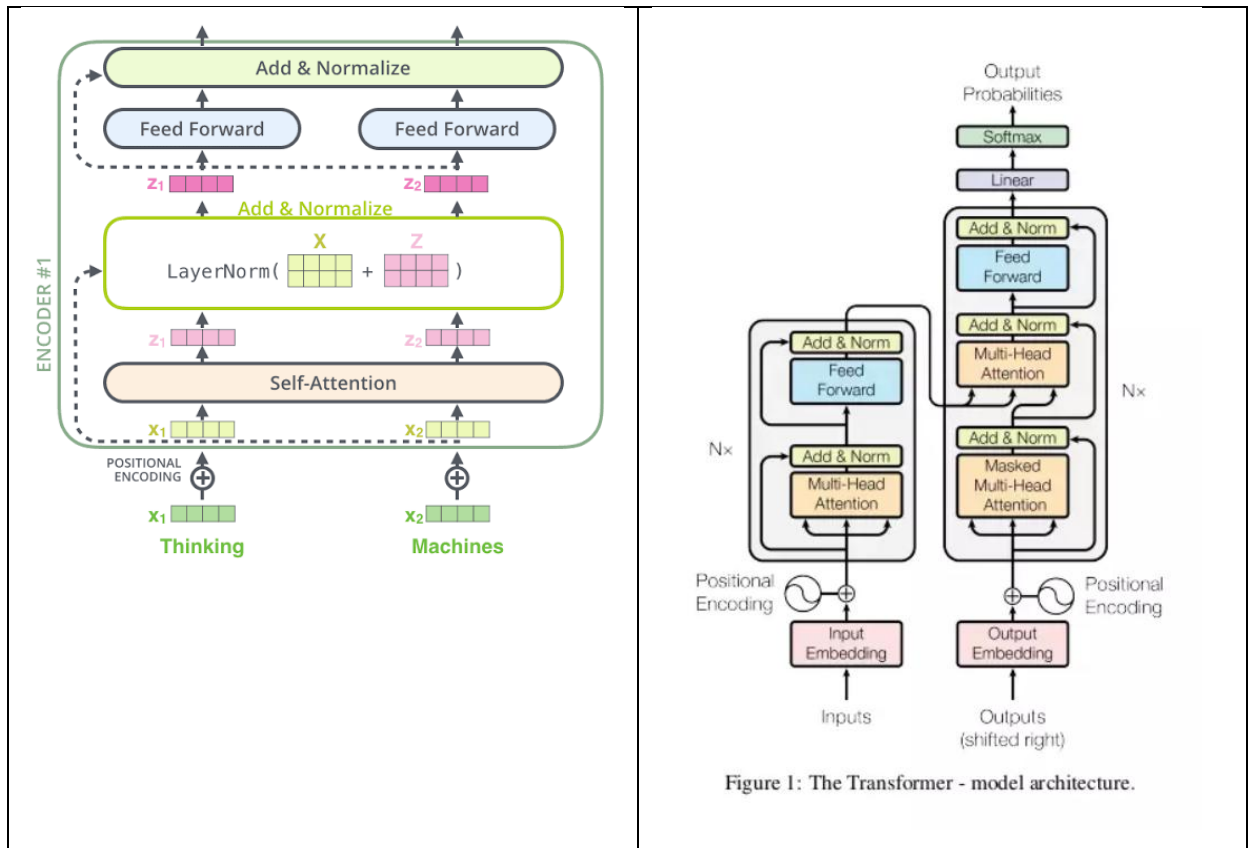
Thành phần quan trọng nhất của Transformer là Self-Attention. Cơ chế này giúp mỗi token trong câu xác định mức độ liên quan với các token còn lại. Công thức Attention được biểu diễn như sau:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó:

- Q là Query.
- K là Key.
- V là Value.
- d_k là số chiều của vector Key.
- softmax dùng để chuẩn hóa trọng số Attention.

Nhờ Self-Attention, Transformer có khả năng hiểu ngữ cảnh tốt hơn, đặc biệt trong các câu dài hoặc các văn bản có nhiều quan hệ ngữ nghĩa phức tạp.



Hình 2: Kiến trúc Transformer và cơ chế Self-Attention.

2.3 Tokenization và biểu diễn văn bản

Tokenization là quá trình chuyển văn bản thô thành các đơn vị nhỏ hơn gọi là token. Sau đó, các token được ánh xạ thành token ID để mô hình có thể xử lý dưới dạng số. Đây là bước tiền xử lý quan trọng trong các bài toán xử lý ngôn ngữ tự nhiên.

Trong bài thực hành này, BERT Tokenizer được sử dụng để xử lý dữ liệu IMDB. Mô hình bert-base-uncased chuyển văn bản về chữ thường và sử dụng phương pháp WordPiece Tokenization. Với phương pháp này, các từ hiếm hoặc từ chưa có trong từ điển có thể được tách thành các subword nhỏ hơn.

Quá trình tokenization của BERT gồm các bước chính: tách văn bản thành token, thêm token đặc biệt [CLS] ở đầu câu, thêm token [SEP] ở cuối câu, chuyển token thành token ID, đồng thời sử dụng padding và truncation để đưa các câu về cùng độ dài.

Biểu diễn đầu vào của Transformer có thể viết như sau:

$$Input\ Embedding = Token\ Embedding + Positional\ Embedding$$

Trong đó, Token Embedding biểu diễn ý nghĩa của token, còn Positional Embedding biểu diễn vị trí của token trong câu.

2.4 Mô hình BERT và Fine-tuning

BERT, viết tắt của Bidirectional Encoder Representations from Transformers, là mô hình ngôn ngữ dựa trên phần Encoder của Transformer. Điểm mạnh của BERT là khả năng học ngữ cảnh hai chiều, nghĩa là mô hình xem xét cả các từ phía trước và phía sau để hiểu ý nghĩa của một từ trong câu.

Trong bài thực hành này, BERT được sử dụng cho bài toán Sentiment Analysis trên tập dữ liệu IMDB. Mỗi đánh giá phim được phân loại thành một trong hai nhãn: tiêu cực hoặc tích cực. Mô hình BERT pretrained được fine-tune lại trên dữ liệu IMDB để thích nghi với nhiệm vụ phân loại cảm xúc.

Fine-tuning là quá trình huấn luyện tiếp một mô hình đã được huấn luyện trước trên một tập dữ liệu cụ thể. Thay vì huấn luyện từ đầu, fine-tuning tận dụng kiến thức ngôn ngữ mà mô hình đã học được, giúp giảm thời gian huấn luyện và cải thiện hiệu quả.

Đầu ra của BERT được đưa qua một lớp phân loại. Sau đó, hàm softmax được sử dụng để chuyển logits thành xác suất:

$$p_i = e^{z_i} / \sum_j e^{z_j}$$

Trong đó:

- z_i là logit của lớp thứ i .
- p_i là xác suất dự đoán của lớp thứ i .

Hàm mất mát thường dùng cho bài toán phân loại là Cross Entropy Loss:

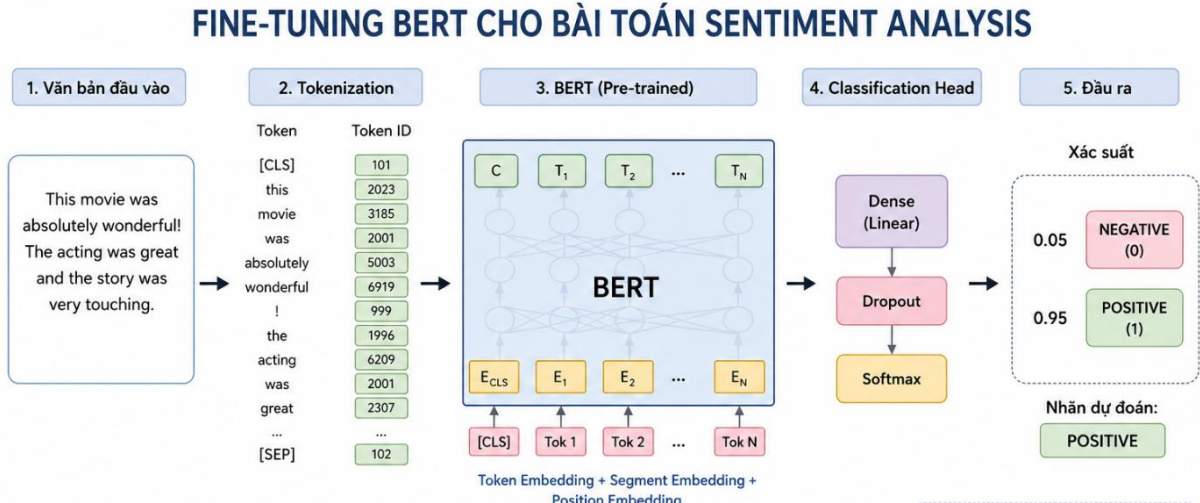
$$L = -(1/N) \sum_n \sum_i y_{n,i} \log(p_{n,i})$$

Trong đó:

- N là số lượng mẫu dữ liệu.
- $y_{n,i}$ là nhãn thật của mẫu thứ n .

- $p_{n,i}$ là xác suất dự đoán của mô hình.

Mục tiêu của quá trình huấn luyện là giảm giá trị loss, từ đó giúp mô hình phân loại cảm xúc chính xác hơn.



Hình 3: Quy trình fine-tuning BERT cho bài toán Sentiment Analysis.

2.5 Sentiment Analysis và chỉ số đánh giá

Sentiment Analysis là bài toán phân tích cảm xúc của văn bản nhằm xác định quan điểm tích cực, tiêu cực hoặc trung lập. Trong bài thực hành này, tập dữ liệu IMDB được dùng để phân loại đánh giá phim thành hai lớp: Negative và Positive.

Quy trình phân loại cảm xúc gồm các bước: văn bản đầu vào được tokenization, đưa vào mô hình BERT, sau đó lớp phân loại dự đoán nhãn cảm xúc tương ứng.

Để đánh giá mô hình, chỉ số Accuracy được sử dụng phổ biến. Accuracy cho biết tỷ lệ dự đoán đúng trên tổng số mẫu:

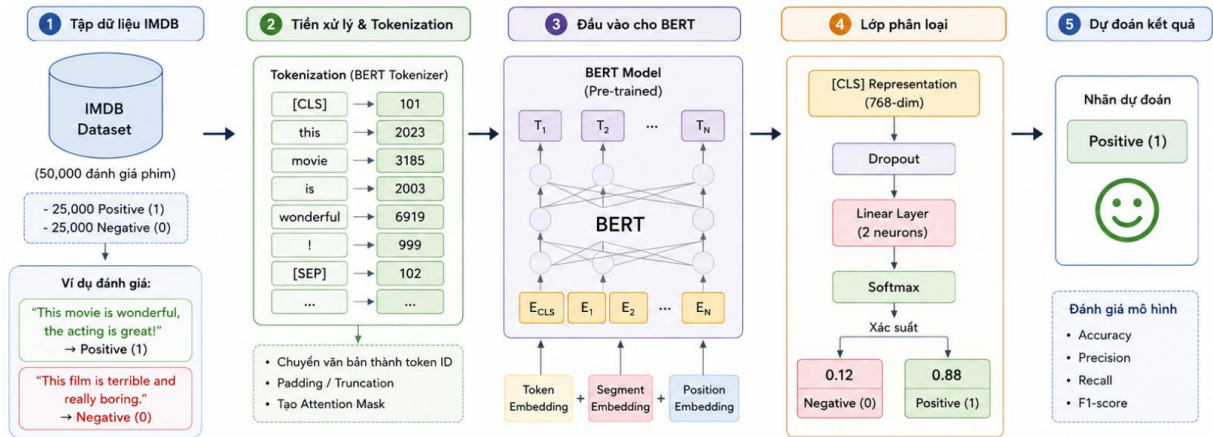
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Trong đó:

- TP là số mẫu tích cực được dự đoán đúng.
- TN là số mẫu tiêu cực được dự đoán đúng.
- FP là số mẫu tiêu cực nhưng dự đoán sai thành tích cực.

- FN là số mẫu tích cực nhưng dự đoán sai thành tiêu cực.

Ngoài Accuracy, có thể sử dụng thêm Precision, Recall và F1-score để đánh giá chi tiết hơn. Tuy nhiên, trong bài thực hành này, Accuracy là chỉ số chính để so sánh kết quả mô hình.



Hình 4: Quy trình phân loại cảm xúc trên tập dữ liệu IMDB.

2.6 GPT-2 và Text Generation

GPT-2 là mô hình sinh văn bản được xây dựng dựa trên phần Decoder của Transformer. Khác với BERT dùng để hiểu văn bản, GPT-2 sinh văn bản theo cơ chế tự hồi quy, tức là dự đoán token tiếp theo dựa trên các token đã xuất hiện trước đó.

Quá trình sinh chuỗi văn bản của GPT-2 được biểu diễn như sau:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^N P(w_i | w_1, \dots, w_{i-1})$$

Trong đó:

- w_i là token thứ i trong chuỗi.
- $P(w_i | w_1, \dots, w_{i-1})$ là xác suất token hiện tại dựa trên các token trước đó.
- n là độ dài chuỗi văn bản.

Khi người dùng cung cấp một prompt, GPT-2 sẽ dự đoán token tiếp theo, sau đó tiếp tục sử dụng token vừa sinh để dự đoán token kế tiếp. Quá trình này lặp lại cho đến khi đạt độ dài tối đa hoặc điều kiện dừng.

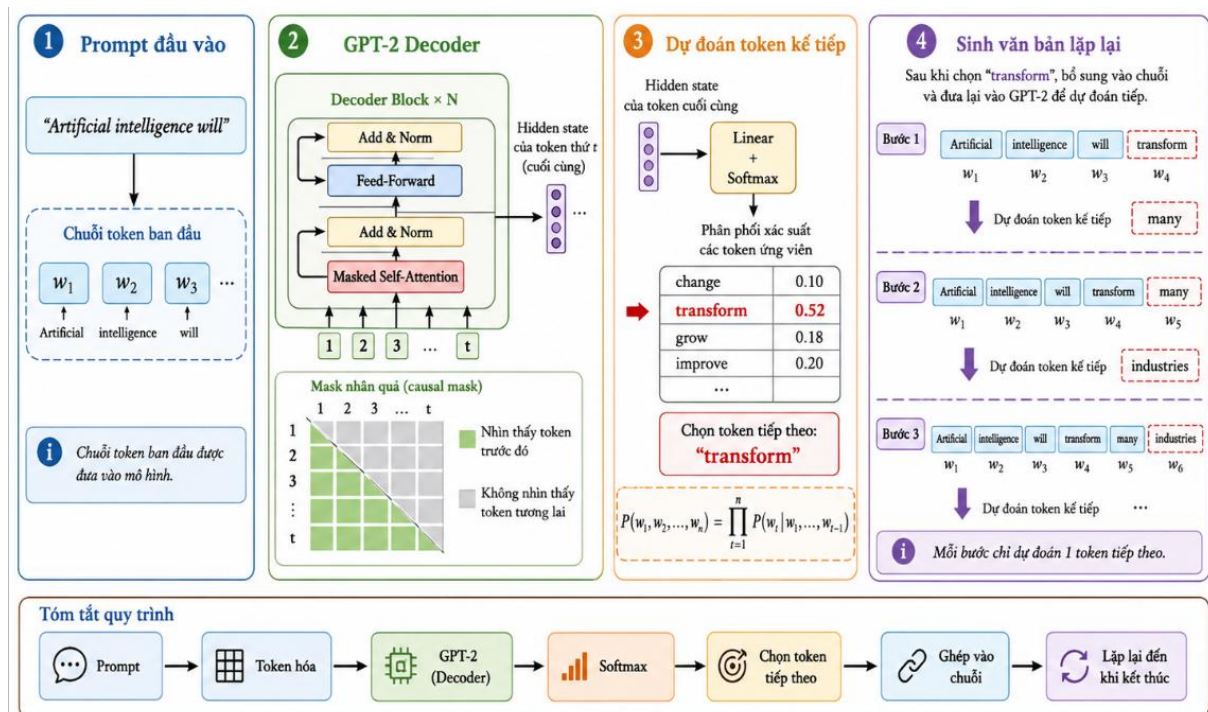
Một số tham số quan trọng trong text generation gồm max_length, num_return_sequences và temperature. Trong đó, temperature ảnh hưởng đến độ ngẫu nhiên của văn bản sinh ra. Giá trị temperature thấp giúp câu trả lời ổn định hơn, còn giá trị cao giúp kết quả đa dạng hơn nhưng dễ thiếu chính xác hơn.

Công thức softmax có temperature:

$$P(w_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Trong đó:

- z_i là logit của token thứ i .
- T là temperature.



Hình 5: Cơ chế sinh văn bản tự hồi quy của GPT-2.

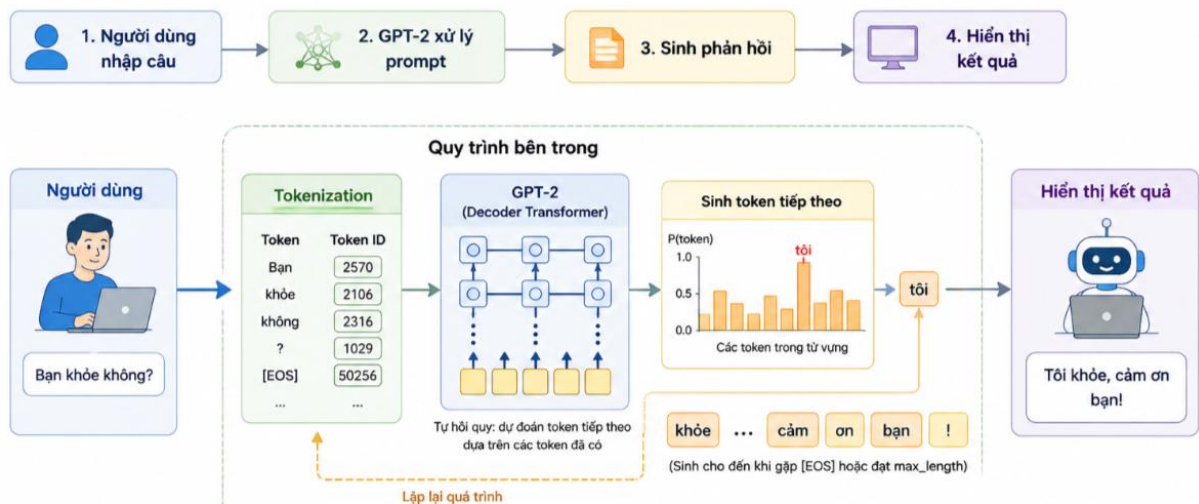
2.7 Prompt Engineering và Chatbot

Prompt Engineering là kỹ thuật thiết kế câu lệnh đầu vào để hướng dẫn mô hình ngôn ngữ tạo ra kết quả phù hợp với yêu cầu. Cùng một nội dung nhưng cách viết prompt khác nhau có thể tạo ra các kết quả khác nhau. Một prompt tốt cần rõ ràng, có ngữ cảnh và thể hiện đúng mục tiêu của người dùng.

Ví dụ, prompt “Explain machine learning” có thể tạo ra câu trả lời chung chung. Trong khi đó, prompt “Explain machine learning in simple terms for beginners” giúp mô hình sinh câu trả lời dễ hiểu hơn cho người mới học.

Trong bài thực hành này, Prompt Engineering được dùng để thử nghiệm các prompt khác nhau với GPT-2 và quan sát sự thay đổi của văn bản được sinh ra. Ngoài ra, GPT-2 cũng được sử dụng để xây dựng chatbot đơn giản. Chatbot nhận câu nhập từ người dùng, đưa câu đó vào mô hình dưới dạng prompt và sinh phản hồi tương ứng.

Quy trình chatbot cơ bản:



Hình 6: Sơ đồ hoạt động của chatbot sử dụng GPT-2.

Tuy nhiên, chatbot đơn giản dùng GPT-2 có thể gặp hạn chế như phản hồi chưa đúng trọng tâm, thiếu kiểm soát hoặc không ghi nhớ tốt ngữ cảnh dài.

2.8 Sentence Transformer, FAISS và RAG

Retrieval-Augmented Generation, viết tắt là RAG, là kỹ thuật kết hợp giữa truy xuất thông tin và sinh văn bản. Thay vì chỉ dựa vào kiến thức đã học trong quá trình huấn luyện, RAG cho phép mô hình tìm kiếm tài liệu liên quan từ nguồn dữ liệu bên ngoài, sau đó sử dụng tài liệu đó để hỗ trợ sinh câu trả lời.

Trong bài thực hành này, Sentence Transformer được dùng để chuyển tài liệu thành vector embedding. Các vector này biểu diễn ý nghĩa của văn bản trong không gian nhiều chiều. Độ tương đồng giữa hai vector có thể được tính bằng Cosine Similarity:

$$\text{cosine_similarity}(x, y) = (x \cdot y) / (||x|| ||y||)$$

Trong đó:

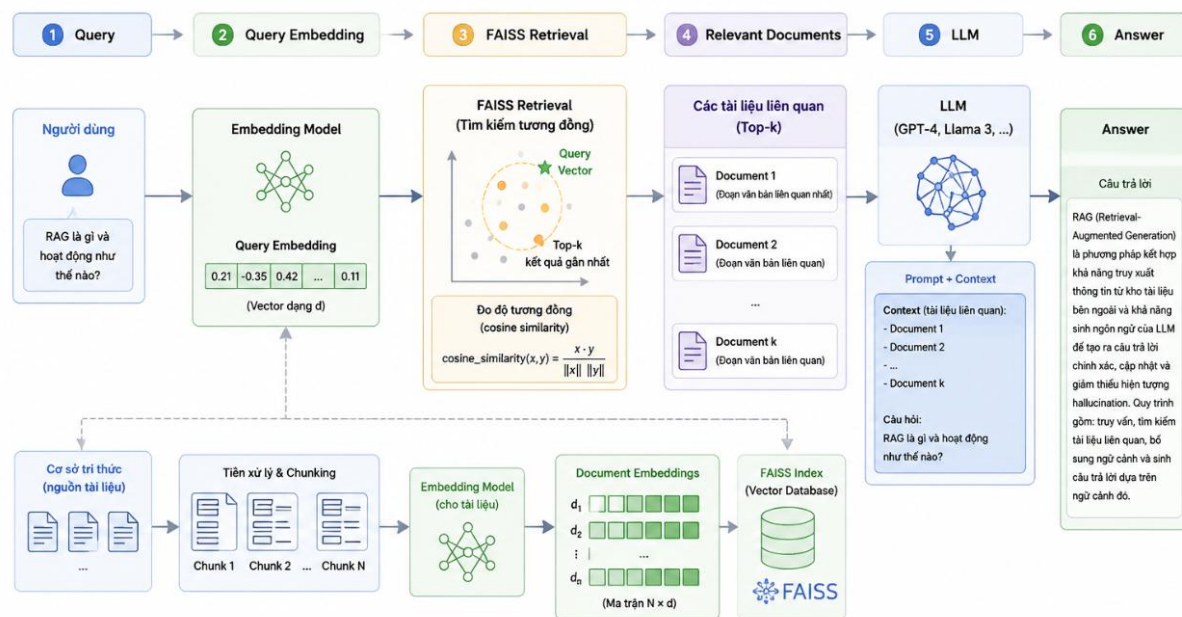
- x và y là hai vector embedding.
- $x \cdot y$ là tích vô hướng giữa hai vector.
- $||x||$ và $||y||$ là độ dài của hai vector.

Sau khi tạo embedding, các vector được lưu vào FAISS index. FAISS giúp tìm kiếm nhanh các vector gần nhất với vector truy vấn. Với khoảng cách Euclidean, công thức được viết như sau:

$$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Trong đó, khoảng cách càng nhỏ thì hai vector càng gần nhau, tức là nội dung văn bản càng tương đồng.

Quy trình RAG gồm các bước chính:



Hình 7: Kiến trúc hệ thống Retrieval-Augmented Generation.

Ưu điểm của RAG là giúp mô hình trả lời dựa trên tài liệu cụ thể, giảm hiện tượng hallucination và dễ cập nhật tri thức mới mà không cần huấn luyện lại mô hình. Vì vậy, RAG phù hợp với các hệ thống hỏi đáp, chatbot và trợ lý AI cần sử dụng dữ liệu riêng hoặc dữ liệu thường xuyên thay đổi.

3. QUY TRÌNH THỰC HIỆN

3.1 Môi trường và thư viện

Notebook được chạy trên Kaggle. Kết quả kiểm tra môi trường ghi nhận PyTorch 2.10.0+cu128, CUDA khả dụng và GPU Tesla T4. Các thư viện chính gồm transformers, datasets, sentence-transformers, faiss-cpu, scikit-learn, pandas, NumPy và Matplotlib.

Bảng 2: Môi trường thực nghiệm

Thành phần	Giá trị / cấu hình	Vai trò
Nền tảng	Kaggle Notebook	Chạy mã nguồn và lưu kết quả
GPU	Tesla T4	Fine-tune BERT và sinh văn bản
PyTorch	2.10.0+cu128	Huấn luyện mô hình
Transformers	AutoTokenizer, Trainer, pipeline	BERT và GPT-2
SentenceTransformer	all-MiniLM-L6-v2	Tạo embedding tài liệu
FAISS	IndexFlatL2	Truy xuất vector gần nhất
Seed	42	Tăng khả năng lặp lại kết quả

3.2 Chuẩn bị và kiểm tra dữ liệu

- Tự động tìm thư mục aclImdb trong /kaggle/input.
- Đọc file .txt từ hai thư mục neg và pos, gán nhãn lần lượt là 0 và 1.
- Giới hạn số mẫu theo từng lớp, ghép thành DataFrame và xáo trộn với seed 42.
- Kiểm tra kích thước dữ liệu và in ba review mẫu để xác nhận văn bản, nhãn.

```
train_df = read_imdb_split("train", max_per_class=1500)
test_df = read_imdb_split("test", max_per_class=500)

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)

train_df.head()
```

Các thư mục trong /kaggle/input:

- /kaggle/input/datasets

Tìm thấy dataset tại: /kaggle/input/datasets/pankrzysiu/keras-imdb/aclImdb_v1/aclImdb

Train shape: (3000, 2)

Test shape: (1000, 2)

	text	label
0	This movie should not be compared to "The Stin...	1
1	The various nudity scenes that other reviewers...	0
2	This movie is amazing! While being funny and e...	1
3	"Don't bother to watch this film" would be bet...	0
4	In September 2003 36-year-old Jonny Kennedy di...	1

Hình 8: Kết quả kiểm tra dữ liệu IMDB sau khi tải vào chương trình

3.3 Tạo Dataset và tokenization

Lớp IMDBTorchDataset đóng gói văn bản, nhãn và tokenizer. Mỗi lần lấy mẫu, văn bản được token hóa, squeeze về vector một chiều và trả về input_ids, attention_mask cùng labels. Kiểm tra mẫu đầu tiên cho kết quả input_ids và attention_mask có kích thước [128], nhãn bằng 1.

3.4 Huấn luyện và đánh giá BERT

Bảng 3: Các siêu tham số của mô hình BERT chính

Tham số	Giá trị	Ý nghĩa
Model	bert-base-uncased	Checkpoint tiền huấn luyện
Số lớp	2	Negative / Positive
Max length	128 token	Độ dài chuỗi cố định
Learning rate	2e-5	Tốc độ cập nhật trọng số
Batch size	8	Train và evaluate
Epoch	1	Số lần duyệt tập train
Weight decay	0,01	Điều chuẩn mô hình
FP16	Có khi CUDA khả dụng	Giảm bộ nhớ, tăng tốc

```
training_args = TrainingArguments(  
    learning_rate=2e-5,
```

```
per_device_train_batch_size=8,  
per_device_eval_batch_size=8,  
num_train_epochs=1,  
weight_decay=0.01,  
eval_strategy="epoch",  
fp16=torch.cuda.is_available()  
)
```

3.5 Thí nghiệm learning rate

Ba mô hình mới được khởi tạo độc lập từ bert-base-uncased và huấn luyện trên cùng tập con 1.000 mẫu train, 300 mẫu test. Các giá trị learning rate lần lượt là $5e-5$, $2e-5$ và $1e-5$; các tham số còn lại được giữ cố định. Thiết kế này giúp so sánh tương đối tác động của learning rate trong giới hạn một epoch.

3.6 Sinh văn bản và xây dựng chatbot

GPT-2 được tải bằng pipeline text-generation. Notebook thử prompt mở đầu Artificial intelligence will, năm prompt khác nhau và ba cấu hình max_length/temperature. Hàm chatbot_reply tạo prompt theo mẫu User/ Bot, sau đó chạy sinh mẫu với temperature 0,8, top_k 50 và top_p 0,95.

3.7 Xây dựng RAG

Tạo kho tri thức gồm 7 câu về machine learning, deep learning, Transformer, BERT, GPT, RAG và sentiment analysis.

Mã hóa tài liệu bằng all-MiniLM-L6-v2, thu được embedding 384 chiều kiểu float32.

Thêm embedding vào FAISS IndexFlatL2.

Mã hóa query, tìm top-k tài liệu có khoảng cách nhỏ nhất và trả về nội dung cùng distance.

Ghép các tài liệu vào prompt Context - Question - Answer rồi gọi GPT-2 để sinh câu trả lời.

4. KẾT QUẢ THỰC NGHIỆM

4.1 Bài 1: Tải dataset IMDB

Yêu cầu của bài 1 là tải bộ dữ liệu IMDB và kiểm tra cấu trúc dữ liệu. Do Kaggle đã được gắn sẵn Large Movie Review Dataset, notebook không gọi load_dataset từ Internet mà tự động tìm thư mục aclImdb trong /kaggle/input, đọc các file văn bản ở hai lớp neg và pos rồi tạo DataFrame gồm hai cột text và label.

```
ACL_ROOT = [p for p in Path("/kaggle/input").rglob("aclImdb") if
p.is_dir()][0]
train_df = read_imdb_split("train", max_per_class=1500)
test_df = read_imdb_split("test", max_per_class=500)
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
```

Kết quả notebook:

Các thư mục trong /kaggle/input:
- /kaggle/input/datasets
Tìm thấy dataset tại: /kaggle/input/datasets/pankrzysiu/keras-imdb/aclImdb_v1/aclImdb
Train shape: (3000, 2)
Test shape: (1000, 2)

	text	label
0	This movie should not be compared to "The Stin...	1
1	The various nudity scenes that other reviewers...	0
2	This movie is amazing! While being funny and e...	1
3	"Don't bother to watch this film" would be bet...	0
4	In September 2003 36-year-old Jonny Kennedy di...	1

Hình 9: Thông tin tập dữ liệu IMDB sau khi tải và các mẫu dữ liệu đầu tiên

Bảng 4: Kết quả tải dataset IMDB từ Kaggle Input

Tập	Số mẫu mỗi lớp	Tổng mẫu	Số cột	Kết quả
Train	1.500 neg + 1.500 pos	3.000	2	Tải thành công
Test	500 neg + 500 pos	1.000	2	Tải thành công

Dữ liệu được tải đúng và cân bằng giữa hai lớp. Việc giới hạn số file giúp thời gian fine-tune phù hợp với GPU Tesla T4 nhưng vẫn giữ đủ mẫu để quan sát khả năng phân loại của BERT.

4.2 Bài 2: Hiển thị một số câu trong dataset

Notebook duyệt ba dòng đầu của train_df, in nhãn số, tên nhãn và tối đa 1.000 ký tự review. Kết quả kiểm tra trực quan cho thấy văn bản và nhãn sentiment tương ứng hợp lý.

```
label_map = {
    0: "Negative",
    1: "Positive"
}

for i in range(3):
    print("=" * 100)
    print("Mẫu số:", i)
    print("Label:", train_df.loc[i, "label"], "-", label_map[train_df.loc[i,
"label"]])
    print("Text:")
    print(train_df.loc[i, "text"][:1000])
```

Kết quả:

```
=====
Mẫu số: 0
Label: 1 - Positive
Text:
This movie should not be compared to "The Sting", or other caper/heist/con game films. What makes it such a great movie experience is what it has to say about relationships, deceit and
=====
Mẫu số: 1
Label: 0 - Negative
Text:
The various nudity scenes that other reviewers referred to are poorly done and a body double was obviously used. If Ms. Pacula was reluctant to do the scenes herself perhaps she should
=====
Mẫu số: 2
Label: 1 - Positive
Text:
This movie is amazing! While being funny and entertaining, it is also profoundly deep and eye-opening. I will watch it again and again. Bruce is a guy who is unhappy with his life. He l
```

Hình 10: Kết quả hiển thị một số câu hỏi trong dataset

Bảng 5: Ba mẫu review được hiển thị từ tập train

Mẫu	Nhãn	Trích đoạn kết quả notebook
0	1 - Positive	This movie should not be compared to 'The Sting'... Highly, hugely recommended!
1	0 - Negative	The various nudity scenes... are poorly done... Canadian movies are generally poorly written and lack entertainment value...
2	1 - Positive	This movie is amazing! While being funny and entertaining,

Mẫu	Nhãn	Trích đoạn kết quả notebook
		it is also profoundly deep and eye-opening...

Mẫu 0 và 2 chứa các từ/cụm tích cực rõ ràng như recommended, amazing, inspirational nên mang nhãn Positive. Mẫu 1 phê bình cảnh quay và chất lượng nội dung nên nhãn Negative là phù hợp. Bước này xác nhận quy trình đọc file không làm lệch nhãn.

4.3 Bài 3: Tokenize dữ liệu bằng BERT tokenizer

bert-base-uncased được dùng để biến mỗi review thành input_ids và attention_mask. Notebook đặt max_length=128, cắt chuỗi dài và padding chuỗi ngắn để mọi mẫu có cùng kích thước.

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
train_dataset = IMDBTorchDataset(train_df["text"], train_df["label"],
tokenizer, 128)
sample = train_dataset[0]
print(sample["input_ids"].shape)
print(sample["attention_mask"].shape)
print(sample["labels"])
```

Kết quả notebook:

```
Số mẫu train: 3000
Số mẫu test: 1000
input_ids shape: torch.Size([128])
attention_mask shape: torch.Size([128])
label: tensor(1)
```

Hình 11: Hiển thị số mẫu train và test

Kết quả đúng với cấu hình: mỗi review tạo thành hai vector dài 128 phần tử và một nhãn vô hướng. Attention mask giúp BERT bỏ qua các vị trí padding trong cơ chế self-attention.

4.4 Bài 4: Fine-tune mô hình BERT

Mô hình AutoModelForSequenceClassification được khởi tạo từ bert-base-uncased với num_labels=2. Trainer huấn luyện toàn bộ 3.000 mẫu trong một epoch, learning rate 2e-5, batch size 8, weight decay 0,01 và fp16 trên GPU.

```

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2
)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
trainer.train()

```

Kết quả notebook:

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok i
 - MISSING :those params were newly initialized because missing from the checkpoin
- `logging\_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD\_LOGGING`
 /usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: We
 return super().apply(*args, **kwargs) # type: ignore[misc]

[188/188 01:04, Epoch 1/1]						
Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.816180	0.719616	0.842000	0.836614	0.850000	0.843254

Hình 12: Kết quả Fine-tuning mô hình BERT

Bảng 6: Kết quả quá trình fine-tune BERT

Thông số đầu ra	Giá trị	Ý nghĩa
Global step	188	Số bước cập nhật sau một epoch
Training loss	0,9851	Loss trung bình do Trainer báo cáo
Runtime	66,8097 giây	Thời gian huấn luyện
Samples/second	44,904	Thông lượng xử lý mẫu

Thông số đầu ra	Giá trị	Ý nghĩa
Steps/second	2,814	Thông lượng bước cập nhật

classifier.weight và classifier.bias được báo MISSING vì checkpoint BERT tiền huấn luyện không có classification head cho IMDB. Hai tham số này được khởi tạo mới và học trong quá trình fine-tune; đây không phải lỗi tải mô hình.

4.5 Bài 5: Tính accuracy trên tập test

Hàm compute_metrics lấy argmax của logits rồi tính accuracy, precision, recall và F1-score cho lớp Positive. Sau khi fine-tune, trainer.evaluate() được chạy trên toàn bộ 1.000 mẫu test.

```
eval_result = trainer.evaluate()

print("Kết quả đánh giá:")
for k, v in eval_result.items():
    print(f"{k}: {v}")
```

Kết quả notebook:

[63/63 00:06]

```
Kết quả đánh giá:
eval_loss: 0.7196162343025208
eval_accuracy: 0.842
eval_precision: 0.8366141732283464
eval_recall: 0.85
eval_f1: 0.8432539682539683
eval_runtime: 7.1336
eval_samples_per_second: 140.182
eval_steps_per_second: 8.831
epoch: 1.0
```

Hình 13: Kết quả accuracy trên tập test

Bảng 7: Kết quả đánh giá BERT trên tập test

Chỉ số	Giá trị	Diễn giải
Eval loss	0,7196	Mất mát trên 1.000 mẫu test
Accuracy	0,8420 (84,20%)	Tỷ lệ dự đoán đúng tổng thể
Precision	0,8366 (83,66%)	Độ chính xác của dự đoán Positive

Chỉ số	Giá trị	Diễn giải
Recall	0,8500 (85,00%)	Tỷ lệ nhận diện đúng mẫu Positive
F1-score	0,8433 (84,33%)	Trung hòa precision và recall
Runtime	7,1336 giây	140,18 mẫu/giây

Notebook còn kiểm tra mô hình với ba câu tự viết. Hai câu có cảm xúc rõ đạt confidence gần 0,89; câu có sắc thái trung tính chỉ đạt 0,6221.

Bảng 8: Kết quả dự đoán sentiment cho ba câu kiểm tra

Câu đầu vào	Dự đoán	Độ tin cậy
This movie was amazing. I really loved the story and the acting.	Positive	0,8852
The film was boring and too long. I do not recommend it.	Negative	0,8937
The movie was okay, not great but not terrible.	Positive	0,6221

Accuracy 84,20% và F1 84,33% cho thấy mô hình học được tín hiệu sentiment khá tốt dù chỉ dùng 3.000 mẫu và một epoch. Confidence thấp ở câu thứ ba phản ánh sự khó khăn với cách diễn đạt pha trộn not great và not terrible.

4.6 Bài 6: Thử thay đổi learning rate

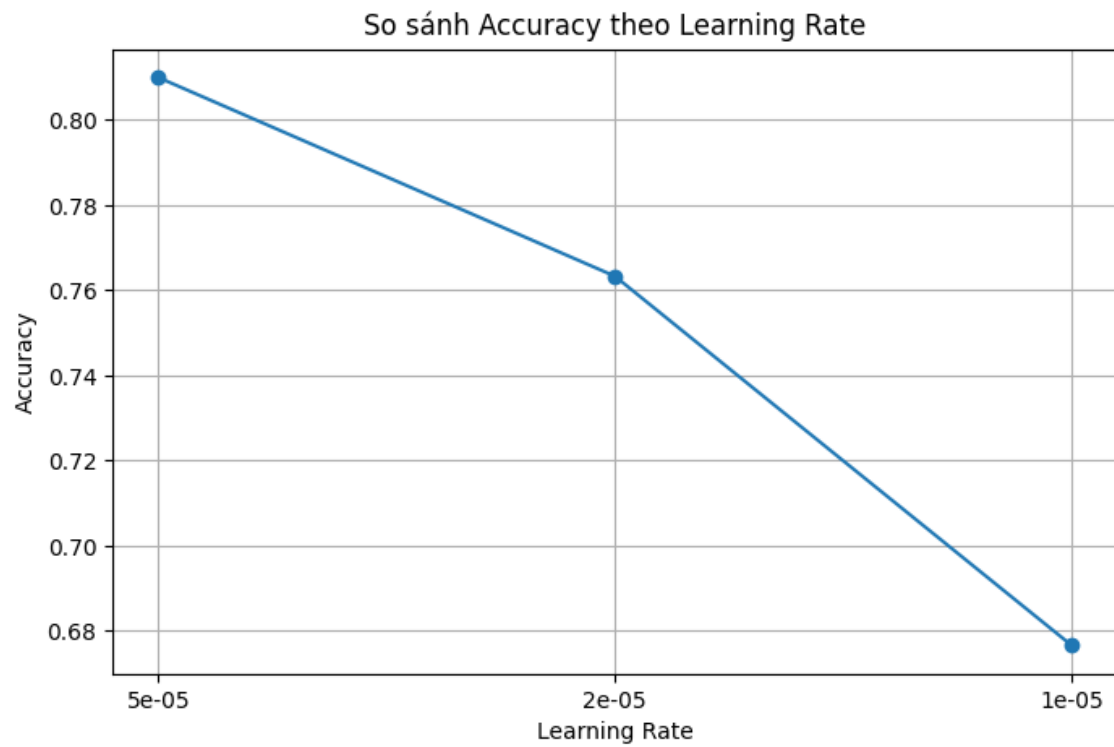
Ba learning rate 5e-5, 2e-5 và 1e-5 được so sánh trên cùng tập con 1.000 mẫu train và 300 mẫu test, mỗi cấu hình khởi tạo lại BERT và chạy một epoch. Các tham số batch size, weight decay và hàm đánh giá được giữ nguyên.

```
learning_rates = [5e-5, 2e-5, 1e-5]
for lr in learning_rates:
    temp_model = AutoModelForSequenceClassification.from_pretrained(
        MODEL_NAME, num_labels=2
    )
    temp_trainer.train()
    result = temp_trainer.evaluate()
```

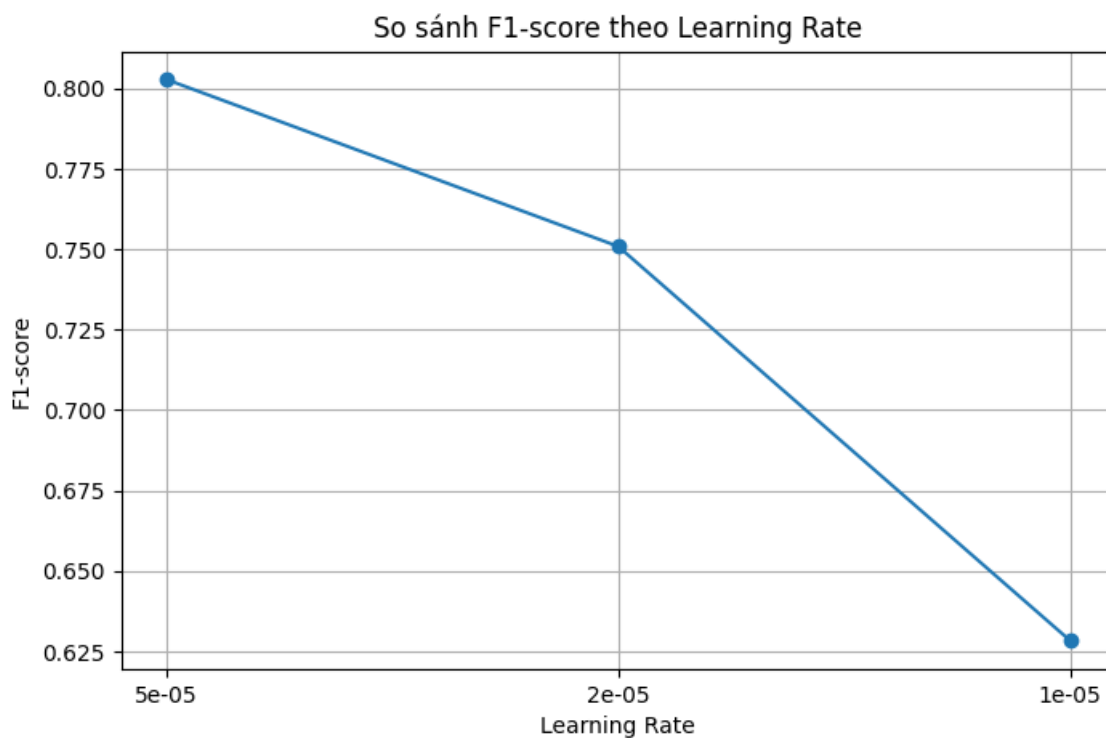
Bảng 9: Kết quả thí nghiệm learning rate trên tập dữ liệu rút gọn

Learning rate	Accuracy	Precision	Recall	F1-score
---------------	----------	-----------	--------	----------

Learning rate	Accuracy	Precision	Recall	F1-score
5e-5	0,8100	0,8000	0,8056	0,8028
2e-5	0,7633	0,7589	0,7431	0,7509
1e-5	0,6767	0,7009	0,5694	0,6284



Hình 14: So sánh Accuracy theo Learning Rate từ notebook



Hình 15: So sánh F1-score theo Learning Rate từ notebook

Learning rate 5e-5 đạt accuracy 81,00% và F1 80,28%, tốt nhất trong thí nghiệm ngắn. Khi giảm xuống 1e-5, recall chỉ còn 56,94%, cho thấy mô hình cập nhật chưa đủ sau một epoch. Kết quả này không được so sánh trực tiếp với accuracy 84,20% của mô hình chính vì kích thước tập huấn luyện khác nhau.

4.7 Bài 7: Text generation với GPT-2

Pipeline text-generation sử dụng GPT-2 để sinh hai chuỗi từ prompt Artificial intelligence will với do_sample=True và temperature=0,8.

```
result = generator(  
    "Artificial intelligence will",  
    max_length=40,  
    num_return_sequences=2,  
    do_sample=True,  
    temperature=0.8  
)
```

Kết quả:

```

=====
Kết quả 1:
Artificial intelligence will come in many forms, from artificial intelligence to artificial intelligence - but all the while
What are some of the most startling discoveries in the history of the human race and of our species?

First, we have discovered a new way of measuring pain, and of controlling the blood supply of animals. It will follow for a
Second, we have discovered that some of the world's greatest animals, all of them of human origin, are capable of suffering
Third, we have discovered that, at some point in time, some of the world's greatest apes, like the great apes known as the
Finally, we have come to understand that the very thing which caused the great apes to vanish was a man named the mad man v
The great ape and its companion animals, who were once in the forest, were now living and flourishing as children.

And now, when we study the great ape
=====
Kết quả 2:
Artificial intelligence will become as ubiquitous as the internet itself, with data collected from our smartphones likely to
What makes this possible?

The real question is whether AI is capable of doing some of the things that we have traditionally thought it couldn't: making

```

Hình 16: Kết quả sinh văn bản bằng mô hình GPT-2

Kết quả 1 từ notebook:

Artificial intelligence will come in many forms, from artificial intelligence to artificial intelligence - but all the while we are going about our day.

What are some of the most startling discoveries in the history of the human race and of our species?...

Kết quả 2 từ notebook:

Artificial intelligence will become as ubiquitous as the internet itself, with data collected from our smartphones likely to be as much as ten times more accurate...

The most important question is what we can expect from AI in the future.

Chuỗi thứ hai bám chủ đề AI tốt hơn và có mạch lập luận về ứng dụng, dữ liệu và ra quyết định. Chuỗi thứ nhất nhanh chóng chuyển sang chủ đề phát hiện khoa học, đau và động vật nên độ liên quan thấp. Cả hai đầu ra đều dài hơn `max_length=40` vì notebook phát sinh cảnh báo `max_new_tokens=256` được ưu tiên hơn `max_length`.

4.8 Bài 8: Thử các prompt và tham số sinh

Notebook thử năm prompt với cùng `temperature=0,9`, `top_k=50`, `top_p=0,95`. Kết quả bên dưới trích trực tiếp phần mở đầu của từng chuỗi sinh.

```

prompt = "Machine learning is"

settings = [
    {"max_length": 40, "temperature": 0.5},
    {"max_length": 60, "temperature": 0.8},
    {"max_length": 80, "temperature": 1.2}
]

for setting in settings:
    print("=" * 100)
    print("Setting:", setting)

    output = generator(
        prompt,
        max_length=setting["max_length"],
        num_return_sequences=1,
        do_sample=True,
        temperature=setting["temperature"],
        top_k=50,
        top_p=0.95
    )

    print(output[0]["generated_text"])

```

```

Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length` (=40) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.
=====
Setting: {'max_length': 40, 'temperature': 0.5}
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length` (=60) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.
Machine learning is still a work in progress, but it's already becoming a reality.

In the next few months, we'll be making a few more changes to our training system to make it easier to learn.

We'll be implementing a new "learning mode" for our training system, where you can train multiple times a day. This mode will allow you to learn from different perspectives.

We'll be implementing a new "learning mode" for our training system, where you can train multiple times a day. This mode will allow you to learn from different perspectives.

We'll be implementing a new "learning mode" for our training system, where you can train multiple times a day. This mode will allow you to learn from different perspectives.

We'll be implementing a new "learning mode" for our training system, where you can train multiple times a day. This mode
=====
Setting: {'max_length': 60, 'temperature': 0.8}
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length` (=80) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.
Machine learning is the most important thing for anyone who wants to understand the world around them. It can give us a great insight into the environment and the way we live.

The key to this is to take the time to understand the world around you and how it is. If you can understand and understand the world around you, then it will help you better understand the world.

Why are we learning?

The answer to this is simple. We're learning to understand the world around us by taking the time to understand the world around us.

But what's the purpose of this?

Well, let's see.

```

Hình 17: Kết quả thử các prompt và tham số sinh

Bảng 10: Kết quả sinh văn bản với năm prompt khác nhau

Prompt	Trích đoạn đầu ra notebook	Đánh giá
The future of AI	...might be more of an exercise in futurism... a kind of deep-learning	Có liên quan AI nhưng diễn đạt vòng

Prompt	Trích đoạn đầu ra notebook	Đánh giá
	machine learning...	
In the year 2050	...the UN predicts that half of the population will be overweight... almost every major country... on the brink of starvation.	Chuyển sang lương thực; lập luận mâu thuẫn
Machine learning is	Machine learning is a core component of the Google Cloud Platform...	Tự gán định nghĩa với Google Cloud
Explain machine learning in simple terms:	The first line of code... Data Mining Protocol... reads and writes data from a MongoDB database.	Không giải thích đúng; tạo khái niệm không có cơ sở
A robot and a human are working together	...to build a giant robot that will walk on water... capable of fighting water and fire.	Có tính kể chuyện nhưng lặp chi tiết

Ngoài thay đổi prompt, notebook giữ prompt Machine learning is và thử ba cặp max_length/temperature để quan sát độ dài, độ đa dạng và mức lặp.

Bảng 11: Quan sát khi thay đổi max_length và temperature

Cấu hình	Kết quả quan sát	Nhận xét
40; 0,5	Lặp nhiều đoạn We'll be implementing a new learning mode...	Ổn định nhưng độ đa dạng thấp
60; 0,8	Có cấu trúc hỏi đáp Why are we learning? nhưng lặp world around us và One is to read.	Cân bằng hơn nhưng vẫn lặp
80; 1,2	Nói về learning algorithms, data structures và AI nhưng lập luận phân tán.	Đa dạng cao, nguy cơ lạc đề lớn

4.9 Bài 9: Xây dựng chatbot đơn giản

Chatbot ghép câu hỏi theo mẫu User: ... Bot: rồi đưa vào GPT-2. Notebook thử tự động ba câu hỏi và tiếp tục thử vòng lặp nhập tay với điều kiện dừng quit/exit/stop.

```
def chatbot_reply(user_input, max_length=80):
    prompt = f"User: {user_input}\nBot:"

    result = generator(
        prompt,
        max_length=max_length,
        num_return_sequences=1,
        do_sample=True,
        temperature=0.8,
        top_k=50,
        top_p=0.95,
        pad_token_id=generator.tokenizer.eos_token_id
    )

    generated = result[0]["generated_text"]
    return generated
```

```
=====
User: Can machine learning help doctors?
Both `max_new_tokens` (=256) and `max_length` (=80) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main\_classes/text\_generation)
User: Can machine learning help doctors?
Bot: yes, we have worked with a number of patients who have had this, but the biggest problem we see is that it's not really possible to actually see what it looks like. So we have to
This is something that is really important to get right, because some people can't get it right. There are people who can, but they don't do it. It's much easier if you have a computer
Bot: right now the main focus is on the brain. And I think that's not a problem.
I'll say this: right now the main focus is on the brain. I think that's probably the biggest problem that we have, and it's really important.
Bot: you've talked about having a little bit of a brain to help you.
Bot: there are a lot of ways in which you can do that. For example, it's very hard to understand in terms of an image. You can't have it and then think about it.
I think that's really important.
=====
User: Explain deep learning in simple words.
User: Explain deep learning in simple words.
Bot: We can talk about how to create deep learning systems, how to use deep learning algorithms to teach people how to understand data, how to do basic computer-based experiments. But
Machine Learning: We're really going to start with a really simple way to train a deep neural network.
Bot: Exactly. It's a whole kind of thing called "deep learning." It's like the big machine learning thing, and then we're going to have deep learning systems that are very good at some
Machine Learning: So you're going to train these neural networks?
Bot: We're going to have the big machine learning systems that are very good at something like the word recognition, but when you combine them, you get a lot of great results. That's j
Machine Learning: This is exactly what you've been doing, right?
Bot: Exactly.
Machine Learning: And if you combine these systems, you can really really solve problems.
Machine Learning: We're going to use machine learning to solve real problems.
Machine Learning: And if you
```

Hình 18: Kết quả xây dựng chatbot đơn giản

Bảng 12: Kết quả thử nghiệm mini chatbot

Câu hỏi	Trích đoạn phản hồi thực tế	Nhận xét
What is artificial intelligence?	My name is Bot... I am the creator of AI! (lặp nhiều lần)	Không trả lời định nghĩa; lặp mạnh
Can machine learning help doctors?	...we have worked with a number of patients... it's much easier if you have a computer...	Có liên quan y tế nhưng dài và vòng vo
Explain deep learning in simple words.	We can talk about how to create deep learning systems... train a deep neural network...	Có nội dung đúng một phần nhưng tự sinh nhiều lượt hội thoại
Can AI replace	I don't know what that means... I	Mâu thuẫn và không

Câu hỏi	Trích đoạn phản hồi thực tế	Nhận xét
humans?	really hope that AI takes over humans...	an toàn về nội dung

Vòng lặp nhập tay hoạt động về mặt chương trình nhưng GPT-2 tiếp tục văn bản thay vì quản lý lượt hội thoại. Mô hình tự sinh thêm nhiều nhãn Bot, lặp và đôi lúc tạo câu trả lời mâu thuẫn. Do đó đây mới là chatbot minh họa pipeline, chưa phù hợp triển khai thực tế.

4.10 Bài 10: Thử nghiệm RAG với ít nhất 5 documents

Notebook sử dụng 7 documents, vượt yêu cầu tối thiểu 5. SentenceTransformer all-MiniLM-L6-v2 tạo embedding và FAISS IndexFlatL2 lưu các vector. Kết quả khởi tạo ghi nhận index có 7 document, mỗi vector gồm 384 chiều.

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

documents = [
    "Machine learning is a subset of artificial intelligence that allows computers to learn from data.",
    "Deep learning uses neural networks with many layers to solve complex problems such as image recognition and speech recognition.",
    "Transformers are neural network architectures widely used in natural language processing tasks.",
    "BERT is a transformer-based model designed for understanding text by looking at context from both directions.",
    "GPT is a transformer-based model designed mainly for generating natural language text.",
    "Retrieval-Augmented Generation combines document retrieval with text generation to answer questions using external knowledge.",
    "Sentiment analysis is a text classification task that identifies whether a sentence expresses positive or negative opinion."
]

embedder = SentenceTransformer("all-MiniLM-L6-v2")

doc_embeddings = embedder.encode(documents)
doc_embeddings = np.array(doc_embeddings).astype("float32")

dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(doc_embeddings)
```

```
print("Số document trong FAISS index:", index.ntotal)
print("Embedding dimension:", dimension)
```

Kết quả notebook:

```
Số document trong FAISS index: 7
Embedding dimension: 384
```

Hình 19: Kết quả tạo với 7 documents và embedding dimension 384

Với query What is BERT used for?, notebook truy xuất top_k=3. Khoảng cách L2 càng nhỏ biểu thị vector càng gần query.

```
Query: What is BERT used for?
=====
Top 1
Distance: 0.7681723
Document: BERT is a transformer-based model designed for understanding text by looking at context from both directions.

Top 2
Distance: 1.5661638
Document: Transformers are neural network architectures widely used in natural language processing tasks.

Top 3
Distance: 1.594387
Document: GPT is a transformer-based model designed mainly for generating natural language text.
```

Hình 20: Kết quả truy xuất tài liệu liên quan bằng FAISS trong hệ thống RAG

Bảng 13: Kết quả truy xuất FAISS cho câu hỏi về BERT

Top	Khoảng cách L2	Tài liệu truy xuất
1	0,7682	BERT is a transformer-based model designed for understanding text by looking at context from both directions.
2	1,5662	Transformers are neural network architectures widely used in natural language processing tasks.
3	1,5944	GPT is a transformer-based model designed mainly for generating natural language text.

Top 1 trả lời đúng trọng tâm và có khoảng cách thấp hơn rõ rệt. Notebook sau đó ghép top-2 documents vào prompt RAG cho câu Why is RAG useful?. Context chứa một câu về BERT và một câu định nghĩa RAG.

Trích đoạn câu trả lời RAG + GPT-2:

```
Both `max_new_tokens` (=256) and `max_length` (=160) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (ht)
Query:
Why is RAG useful?
=====
Retrieved context:
BERT is a transformer-based model designed for understanding text by looking at context from both directions.
Retrieval-Augmented Generation combines document retrieval with text generation to answer questions using external knowledge.
=====
Generated answer:

Use the following context to answer the question.

Context:
BERT is a transformer-based model designed for understanding text by looking at context from both directions.
Retrieval-Augmented Generation combines document retrieval with text generation to answer questions using external knowledge.

Question:
Why is RAG useful?

Answer:

RAG is a high-performance, high-value, high-value, high-value (RAG) data-processing, and data-processing system. It is an implementation of the RAG algorithm for data-proces

The RAG model is designed to be highly scalable, efficient, and performable. It is scalable, efficient, and performable.

The RAG model is designed to be highly scalable, efficient, and performable. It is scalable, efficient, and performable.
...
Example:
> RAG = RAG.
```

Hình 21: Kết quả sinh câu trả lời của hệ thống RAG dựa trên tài liệu truy xuất

RAG is a high-performance, high-value, high-value, high-value (RAG) data-processing...

The RAG model is designed to be highly scalable, efficient, and performable...

RAG can be implemented as an independent, parallel, or parallel-processed system...

Retrieval hoạt động, nhưng phần generation lặp và không giải thích đúng lợi ích của RAG. Nguyên nhân chính là GPT-2 cơ bản không được instruction-tune và context top-2 còn chứa một câu BERT không liên quan trực tiếp.

4.11 Bài 11: Phân tích ưu điểm của RAG so với LLM thuần

Notebook nêu năm ưu điểm của RAG: dùng dữ liệu ngoài, giảm hallucination, cập nhật tri thức dễ, phù hợp dữ liệu riêng và hiển thị nguồn để kiểm tra. Các nhận định này được tổng hợp với kết quả thực nghiệm như sau:

Sử dụng dữ liệu bên ngoài: câu trả lời có thể dựa trên tài liệu không nằm trong trọng số LLM.

Giảm hallucination khi tài liệu truy xuất đúng và mô hình sinh tuân thủ chặt context.

Cập nhật tri thức bằng cách thay đổi kho tài liệu hoặc FAISS index, không cần huấn luyện lại toàn bộ LLM.

Phù hợp dữ liệu nội bộ như giáo trình, báo cáo, quy định công ty hoặc dữ liệu hệ thống.

Có khả năng giải thích nguồn vì hệ thống biết document nào đã được truy xuất.

Bảng 14: So sánh RAG và LLM thuần từ kết quả thực nghiệm

Tiêu chí	LLM thuần	RAG	Liên hệ notebook
Nguồn tri thức	Chủ yếu từ trọng số đã huấn luyện	Kết hợp kho tài liệu bên ngoài	FAISS truy xuất đúng câu mô tả BERT
Cập nhật	Thường cần fine-tune/huấn luyện lại	Cập nhật documents và index	Có thể thêm câu mới vào 7 documents
Hallucination	Khó kiểm soát	Có thể giảm nếu bám context	GPT-2 vẫn hallucinate vì không theo chỉ dẫn
Khả năng kiểm tra	Không luôn chỉ ra nguồn	Có thể hiển thị top-k documents	Notebook in document và distance
Điểm yếu	Tri thức có thể cũ	Phụ thuộc retrieval và generator	Context nhiều + GPT-2 lặp làm câu trả lời kém

Kết luận bài 11: Kết quả notebook chứng minh retrieval là điều kiện cần nhưng chưa đủ. RAG chỉ giảm hallucination khi tài liệu liên quan, prompt buộc mô hình trả lời theo context và generator có khả năng làm theo chỉ dẫn. Với GPT-2 hiện tại, cần reranking tài liệu và thay bằng mô hình instruction-tuned để ưu điểm của RAG thể hiện rõ.

5. PHÂN TÍCH VÀ THẢO LUẬN

5.1 Đánh giá mô hình BERT

Accuracy 84,20% và F1 84,33% cho thấy BERT đã học được đặc trưng sentiment tốt chỉ sau một epoch trên 3.000 mẫu. Precision 83,66% thấp hơn recall 85,00% một chút, nghĩa là mô hình nhận diện phần lớn review Positive nhưng vẫn có một tỷ lệ nhất định mẫu Negative bị dự đoán thành Positive. Chênh lệch nhỏ giữa precision và recall cho thấy kết quả tương đối cân bằng.

Kết quả chưa đạt mức tối đa vì tập huấn luyện chỉ chiếm một phần nhỏ của IMDB đầy đủ, chuỗi bị cắt ở 128 token và chỉ chạy một epoch. Nhiều review phim dài có thể chứa thông tin quan trọng ở phần cuối; truncation làm mất ngữ cảnh này. Ngoài ra, câu mơ hồ hoặc pha trộn khen/chê làm confidence giảm, như trường hợp câu kiểm tra thứ ba.

5.2 Ảnh hưởng của learning rate

Trong thí nghiệm rút gọn, learning rate $5e-5$ vượt $2e-5$ khoảng 4,67 điểm phần trăm accuracy và vượt $1e-5$ khoảng 13,33 điểm. Nguyên nhân hợp lý là ngân sách chỉ có một epoch: learning rate nhỏ khiến classifier và các lớp BERT chưa cập nhật đủ. Tuy nhiên, không thể kết luận $5e-5$ luôn tốt nhất; nếu tăng số epoch, giá trị lớn hơn có thể gây dao động hoặc overfitting, còn $2e-5$ có thể hội tụ ổn định hơn.

Mô hình chính dùng $2e-5$ nhưng đạt accuracy 84,20% vì được huấn luyện trên 3.000 mẫu, trong khi bảng learning rate chỉ dùng 1.000 mẫu. Sự khác nhau về kích thước dữ liệu là biến nhiễu quan trọng; báo cáo không dùng hai kết quả này để khẳng định $5e-5$ tốt hơn mô hình chính.

5.3 Prompt engineering và chất lượng GPT-2

Các kết quả cho thấy prompt ảnh hưởng mạnh đến chủ đề, nhưng prompt ngắn không đủ để kiểm soát độ chính xác. GPT-2 sinh câu trôi chảy nhờ khả năng mô hình hóa ngôn ngữ, song dễ chuyển chủ đề, lặp và tạo thông tin không có nguồn. Temperature 0,5 giảm đa dạng nhưng tăng lặp; temperature 1,2 làm văn bản phong phú hơn nhưng thiếu tập trung. Cấu hình 0,8 là phương án trung gian trong notebook, dù vẫn chưa giải quyết được hallucination.

Để thí nghiệm đúng ý nghĩa, cần loại bỏ xung đột giữa `max_new_tokens` và `max_length`. Nên dùng `max_new_tokens` để kiểm soát số token sinh thêm, kết hợp `repetition_penalty`, `no_repeat_ngram_size` và `early_stopping`. Mỗi cấu hình cũng nên chạy nhiều seed rồi đánh giá theo tiêu chí coherence, relevance và factuality thay vì chỉ quan sát một mẫu ngẫu nhiên.

5.4 Hạn chế của chatbot

Chatbot hiện tại chỉ là text continuation, không phải mô hình hội thoại chuyên dụng. Prompt User/ Bot có thể khiến GPT-2 tự sinh thêm nhiều lượt, trong khi hàm không cắt đầu ra tại nhãn User tiếp theo. Mô hình cũng không lưu lịch sử theo cấu trúc, không kiểm soát nội dung và không có bước kiểm chứng. Do đó, phản hồi dài, lặp hoặc mâu thuẫn là kết quả có thể dự đoán.

Có thể cải thiện bằng cách dùng mô hình instruction/chat, chuyển sang `max_new_tokens`, đặt stop sequence theo lượt, chỉ lấy phần sau Bot:, giới hạn lịch sử, thêm system prompt và áp dụng bộ lọc an toàn. Với câu hỏi kiến thức, nên kết hợp RAG và yêu cầu trích nguồn thay vì dùng sinh văn bản thuần.

5.5 Phân tích RAG so với LLM thuần

Truy cập tri thức bên ngoài: kho tài liệu có thể chứa dữ liệu mới hoặc dữ liệu nội bộ không có trong trọng số mô hình.

Giảm hallucination khi mô hình sinh bám sát tài liệu truy xuất và prompt yêu cầu trả lời theo nguồn.

Cập nhật tri thức nhanh: chỉ cần thay đổi tài liệu hoặc index, không nhất thiết huấn luyện lại LLM.

Tăng khả năng kiểm tra: hệ thống có thể hiển thị tài liệu được truy xuất cùng câu trả lời.

Phù hợp miền chuyên biệt: giáo trình, báo cáo, quy định và tài liệu doanh nghiệp có thể được lập chỉ mục riêng.

Thực nghiệm cũng cho thấy giới hạn: query `Why is RAG useful?` truy xuất thêm tài liệu BERT không trực tiếp trả lời câu hỏi, và GPT-2 vẫn bịa nội dung dù đã có câu định nghĩa RAG. Cần nâng chất lượng embedding/kho tài liệu, chuẩn hóa khoảng

cách, thử cosine similarity, reranking và dùng mô hình instruction-tuned để khai thác context.

5.6 Các lỗi gặp phải và cách khắc phục

Bảng 15: Tổng hợp lỗi/cảnh báo trong notebook và hướng khắc phục

Hiện tượng	Nguyên nhân	Cách khắc phục đề xuất
classifier.weight/bias MISSING	Classification head mới khi chuyển từ BERT tiền huấn luyện	Đây là cảnh báo bình thường; cần fine-tune head trên dữ liệu đích
max_new_tokens và max_length cùng được đặt	Generation config xung đột với tham số gọi pipeline	Chỉ dùng max_new_tokens hoặc xóa giá trị mặc định xung đột
GPT-2 lặp nhiều	Mô hình cơ bản, sampling và không có ràng buộc lặp	Thêm repetition_penalty, no_repeat_ngram_size, stop sequence
Chatbot sinh nhiều lượt giả	Prompt User/Bot được tiếp tục như văn bản	Cắt tại nhãn User tiếp theo hoặc dùng chat model
RAG sinh sai dù retrieval đúng	GPT-2 không instruction-tuned và context còn nhiều	Dùng mô hình QA/chat, prompt chặt, rerank tài liệu và trích nguồn
faiss-cpu phải cài trong notebook	Môi trường Kaggle chưa có gói	Cài đầu notebook và bật Internet khi cần

5.7 Đề xuất cải thiện thực nghiệm

Huấn luyện BERT với toàn bộ IMDB hoặc tăng số epoch lên 2-3, dùng validation riêng và early stopping.

Thử max_length 256/512, gradient accumulation và learning-rate scheduler; báo cáo confusion matrix và sai số theo loại review.

Thực hiện grid search learning rate trên cùng tập dữ liệu và lặp lại với nhiều seed để có trung bình, độ lệch chuẩn.

Thay GPT-2 bằng mô hình instruction-tuned phù hợp tài nguyên; kiểm soát max_new_tokens và các tham số chống lặp.

Đánh giá text generation bằng tiêu chí định tính có thang điểm hoặc chỉ số phù hợp, không chỉ chép đầu ra.

Mở rộng kho RAG, chia đoạn tài liệu, dùng cosine similarity, reranker và yêu cầu câu trả lời kèm tài liệu nguồn.

Tách retrieval evaluation và generation evaluation để xác định lỗi thuộc bước nào.

6. KẾT LUẬN

Bài thực hành đã triển khai đầy đủ các nội dung chính về LLM cho NLP: chuẩn bị dữ liệu IMDB, tokenization, fine-tune BERT, đánh giá mô hình, sinh văn bản bằng GPT-2, prompt engineering, chatbot và RAG. Mô hình BERT chính đạt accuracy 84,20%, precision 83,66%, recall 85,00% và F1-score 84,33% trên 1.000 mẫu test. Kết quả dự đoán câu mới cũng phản ánh đúng mức độ chắc chắn giữa câu rõ cảm xúc và câu mơ hồ.

Thí nghiệm learning rate cho thấy $5e-5$ phù hợp nhất trong cấu hình rút gọn một epoch, nhưng kết luận chỉ có giá trị trong phạm vi tập con. Phần GPT-2 chứng minh prompt và temperature thay đổi đáng kể đầu ra, đồng thời bộc lộ hiện tượng lạc đề, lặp và hallucination. Chatbot đơn giản hoạt động về mặt kỹ thuật nhưng chưa đủ tin cậy cho hội thoại thực tế.

RAG truy xuất đúng tài liệu liên quan đến BERT, song GPT-2 chưa sử dụng context tốt cho câu hỏi về lợi ích của RAG. Kết quả này nhấn mạnh rằng một hệ RAG hiệu quả cần đồng thời tối ưu kho tài liệu, retrieval, prompt và mô hình sinh. Hướng phát triển phù hợp là dùng mô hình instruction-tuned, kiểm soát sinh chặt chẽ, mở rộng đánh giá và bổ sung cơ chế trích nguồn.

TÀI LIỆU THAM KHẢO

- [1] Bộ môn Điện tử - Thông tin, Bài thực hành AI 08: Large Language Models for Natural Language Processing, tháng 03/2026.
- [2] Trương Phúc Thịnh, Notebook b-i-8.ipynb: BERT sentiment analysis, GPT-2, chatbot và RAG, thực hiện trên Kaggle, tháng 06/2026.
- [3] Hugging Face Transformers, các API AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer và pipeline được sử dụng trong notebook.
- [4] Sentence Transformers và FAISS, các thành phần tạo embedding và tìm kiếm vector được sử dụng trong phần RAG.

PHỤ LỤC: MÃ NGUỒN TRỌNG TÂM

A. Hàm tính chỉ số đánh giá

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    acc = accuracy_score(labels, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average="binary"
    )
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

B. Hàm truy xuất tài liệu

```
def retrieve_documents(query, top_k=2):
    query_embedding = embedder.encode([query])
    query_embedding = np.array(query_embedding).astype("float32")
    distances, indices = index.search(query_embedding, top_k)

    return [
        {"document": documents[idx], "distance": dist}
        for idx, dist in zip(indices[0], distances[0])
    ]
```

C. Hàm tạo câu trả lời RAG

```
def rag_answer(query, top_k=2):
```

```

retrieved_docs = retrieve_documents(query, top_k=top_k)
context = "\n".join(item["document"] for item in retrieved_docs)
prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
output = generator(
    prompt,
    max_new_tokens=80,
    do_sample=True,
    temperature=0.7,
    top_k=50,
    top_p=0.95
)
return output[0]["generated_text"]

```